

Sklearn 简介

Sklearn，全称为 **Sklearn**，是一个基于 Python 的开源机器学习库。

Sklearn 是一个功能强大、易于使用的机器学习库，提供了从数据预处理到模型评估的一整套工具。

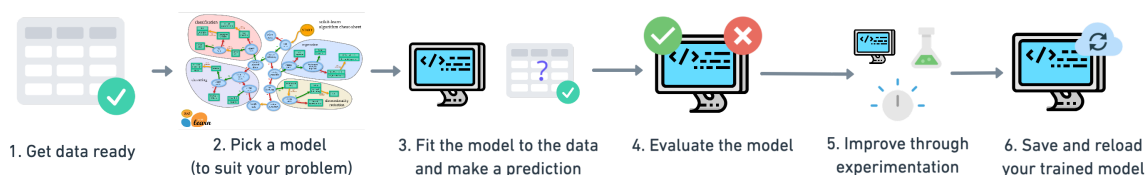
Sklearn 构建在 NumPy 和 SciPy 基础之上，因此它能够高效地处理数值计算和数组操作。

Sklearn 适用于各类机器学习任务，如分类、回归、聚类、降维等。

Sklearn 凭借其简洁而一致的 API，Sklearn 已经成为了机器学习爱好者和专家的必备工具之一。

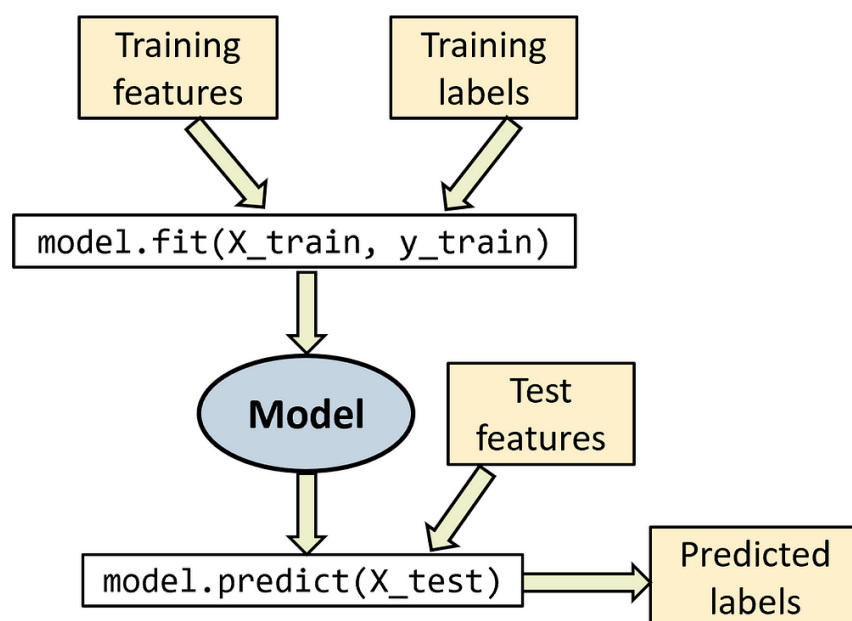
Sklearn 如何工作

在 Sklearn 中，机器学习的流程遵循一定的模式：数据加载、数据预处理、训练模型、评估模型 和 调优模型。



具体工作流程如下：

- **数据加载**：使用 Sklearn 或其他库加载数据集，例如通过 `datasets.load_iris()` 加载经典的鸢尾花数据集，或使用 `train_test_split()` 分割数据。
- **数据预处理**：根据数据的类型，可能需要进行标准化、去噪、缺失值填充等操作。
- **选择算法和训练模型**：选择适合的算法（如逻辑回归、支持向量机等），使用 `.fit()` 方法对模型进行训练。
- **模型评估**：使用交叉验证或单一训练/测试集来评估模型的准确性、召回率、F1分数等性能指标。
- **模型优化**：使用网格搜索（`GridSearchCV`）或随机搜索（`RandomizedSearchCV`）对模型进行超参数优化，提高模型性能。



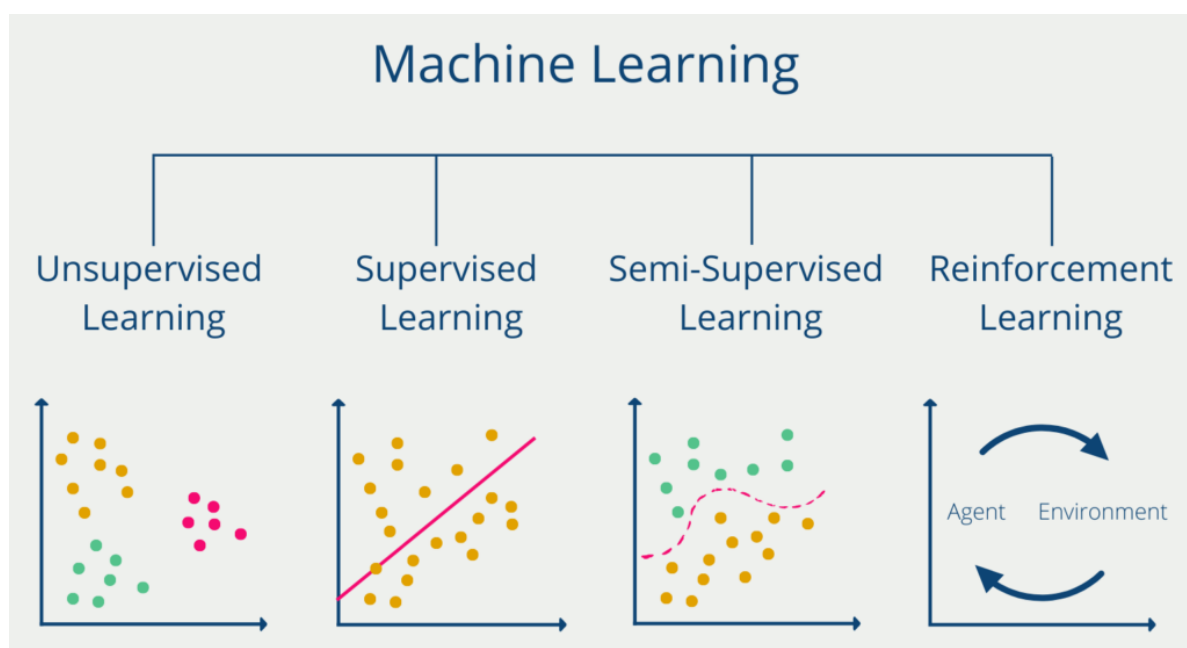
Sklearn 的特点

- **易用性**：Sklearn 的 API 设计简洁且一致，使得学习曲线较为平缓。通过简单的 `fit`、`predict`、`score` 等方法，可以快速实现机器学习任务。
- **高效性**：虽然 Sklearn 是纯 Python 编写，但它的底层大多数实现依赖于 Cython 和 NumPy，这使得它在执行机器学习算法时速度非常快。
- **丰富的功能**：Sklearn 提供了大量经典的机器学习算法，包括：
 - **分类算法**：如逻辑回归、支持向量机 (SVM)、K近邻 (KNN)、随机森林等
 - **回归算法**：如线性回归、岭回归、Lasso回归等
 - **聚类算法**：如 K均值、层次聚类、DBSCAN 等
 - **降维算法**：如主成分分析 (PCA)、t-SNE 等
 - **模型选择与评估**：交叉验证、网格搜索、模型评估指标等
- **兼容性好**：Sklearn 可以很好地与 NumPy、SciPy、Pandas 等 Python 数据处理库兼容，支持多种数据格式 (如 NumPy 数组、Pandas DataFrame) 进行输入和输出。

Sklearn 支持的机器学习任务

Sklearn 提供了丰富的工具，支持以下几类机器学习任务：

- **监督学习 (Supervised Learning)**：
 - **分类问题**：预测数据的类别 (例如，电子邮件垃圾邮件分类、图像分类、疾病预测等)。
 - **回归问题**：预测连续值 (例如，房价预测、股票价格预测等)。
- **无监督学习 (Unsupervised Learning)**：
 - **聚类问题**：将数据分组为不同的类群 (例如，客户分群、文档聚类等)。
 - **降维问题**：将高维数据投影到低维空间，便于可视化或减少计算复杂度 (例如，PCA、t-SNE)。
- **半监督学习 (Semi-supervised Learning)**：部分数据是带标签的，部分数据是无标签的，模型尝试从这些数据中提取信息。
- **强化学习 (Reinforcement Learning)**：虽然 Sklearn 主要专注于监督和无监督学习，但也有一些相关的工具，可以用来处理强化学习问题。



Sklearn 中常见的模块和类

- **分类 (Classification)**
 - `sklearn.linear_model.LogisticRegression`: 逻辑回归
 - `sklearn.svm.SVC`: 支持向量机分类
 - `sklearn.neighbors.KNeighborsClassifier`: K近邻分类
 - `sklearn.ensemble.RandomForestClassifier`: 随机森林分类
- **回归 (Regression)**
 - `sklearn.linear_model.LinearRegression`: 线性回归
 - `sklearn.linear_model.Ridge`: 岭回归
 - `sklearn.ensemble.RandomForestRegressor`: 随机森林回归
- **聚类 (Clustering)**
 - `sklearn.cluster.KMeans`: K均值聚类
 - `sklearn.cluster.DBSCAN`: 基于密度的空间聚类
- **降维 (Dimensionality Reduction)**
 - `sklearn.decomposition.PCA`: 主成分分析 (PCA)
 - `sklearn.decomposition.NMF`: 非负矩阵分解
- **模型选择 (Model Selection)**
 - `sklearn.model_selection.train_test_split`: 将数据集划分为训练集和测试集
 - `sklearn.model_selection.GridSearchCV`: 网格搜索, 寻找最佳超参数
- **数据预处理 (Preprocessing)**
 - `sklearn.preprocessing.StandardScaler`: 标准化
 - `sklearn.preprocessing.MinMaxScaler`: 最小-最大标准化
 - `sklearn.preprocessing.OneHotEncoder`: 独热编码

常用术语解释

- **拟合 (Fit)**: 指将模型应用于训练数据, 并通过训练调整模型的参数。 `model.fit(X_train, y_train)`
- **预测 (Predict)**: 根据训练好的模型, 对未知数据进行预测。 `model.predict(X_test)`
- **评估 (Score)**: 评估模型的性能, 通常返回一个评分指标, 例如准确率。 `model.score(X_test, y_test)`
- **交叉验证 (Cross-validation)**: 将数据集划分为多个子集, 通过多次训练和验证, 评估模型的稳定性和泛化能力。

Sklearn 与其他库的关系

- **与 NumPy 和 SciPy 的关系**: Sklearn 构建在 NumPy 和 SciPy 基础之上, 因此它能够高效地处理数值计算和数组操作。
- **与 Pandas 的关系**: Pandas 提供了强大的数据处理能力, 而 Sklearn 支持从 Pandas 的 DataFrame 中直接提取数据进行模型训练和预测。
- **与 TensorFlow 和 PyTorch 的关系**: Sklearn 主要关注传统的机器学习方法, 而 TensorFlow 和 PyTorch 则更侧重于深度学习模型。尽管如此, Sklearn 与这些库可以结合使用, 处理一些前期的特征工程任务, 或作为基础模型与深度学习进行比较。

Sklearn 安装

要学习 Sklearn, 安装是第一步, 由于 Sklearn 依赖于其他几个库 (如 NumPy、SciPy 和 matplotlib), 所以我们需要确保这些依赖库也已安装。

系统环境要求：

- **Python 版本**：scikit-learn 支持 Python 3.7 及以上版本。
- **操作系统**：scikit-learn 可以在 Windows、macOS 和 Linux 等主流操作系统上运行。
- **包管理工具**：你可以使用 `pip` 或 `conda` 来安装 scikit-learn。

本章节我们使用 `pip` 安装 scikit-learn。

安装前，确保已安装 Python 和 `pip`。

检查 Python 是否安装：

```
1 | python --version
```

检查 `pip` 是否安装：

```
1 | pip --version
```

如果未安装 python 和 `pip`，可以参考我们的：[Python 安装](#) 与 [Pip 安装](#)。

注意：目前最新的 Python 版本已经预装了 `pip`。

注意：Python 2.7.9 + 或 Python 3.4+ 以上版本都自带 `pip` 工具。

安装 Sklearn

Sklearn 全称为 scikit-learn。

使用 `pip` 安装最新版本的 scikit-learn：

```
1 | pip install scikit-learn
```

如果你希望安装特定版本的 scikit-learn，可以指定版本号：

```
1 | pip install scikit-learn==1.2.0
```

检查安装是否成功

安装完成后，我们可以通过以下代码检查 scikit-learn 是否安装成功：

```
1 | import sklearn
2 | print(sklearn.__version__)
```

如果成功显示 scikit-learn 的版本号，类似如下，则表示安装成功：

```
1 | 1.5.2
```

使用 conda 安装 scikit-learn

如果你使用的是 Anaconda 环境，推荐使用 `conda` 来安装 scikit-learn。

Anaconda 是一个用于科学计算的 Python 发行版，内置了许多数据科学和机器学习的库，方便开发者使用。

如果你还不了解 Anaconda，可以参考：[Anaconda 教程](#)。

创建一个新的 conda 环境（可选）

你可以选择为 scikit-learn 创建一个新的虚拟环境，以避免与其他项目发生冲突：

```
1 | conda create -n sklearn-env python=3.9
2 | conda activate sklearn-env
```

安装 scikit-learn

使用 conda 安装 scikit-learn：

```
1 | conda install scikit-learn
```

如果你想安装特定版本，可以指定版本号：

```
1 | conda install scikit-learn=1.2.0
```

验证安装

在 conda 环境中，你可以通过 Python shell 或 Jupyter Notebook 来验证安装：

```
1 | import sklearn
2 | print(sklearn.__version__)
```

如果成功显示 scikit-learn 的版本号，类似如下，则表示安装成功：

```
1 | 1.5.2
```

安装其他依赖

scikit-learn 依赖于一些其他库，特别是：

- **NumPy**：用于处理数组和数值计算
- **SciPy**：提供更高级的数学计算工具
- **matplotlib**（可选）：用于数据可视化
- **joblib**（可选）：用于模型的持久化（保存和加载）

如果你使用 pip 安装，scikit-learn 会自动安装这些依赖，但如果你想手动安装或更新它们，可以使用以下命令：

```
1 | pip install numpy scipy matplotlib joblib
```

使用 conda 安装，则所有的依赖库会自动安装。

Sklearn 基础概念

在使用 Sklearn 进行机器学习时，需要了解一些基础的概念。

Sklearn 提供了一种统一且简洁的 API 来实现各种机器学习算法和流程，能够帮助我们快速实现各种机器学习任务。

接下来我们从以下几个概念展开说明：**数据表示、模型类型、预处理方法、评估指标、模型调优等**。

1、数据表示：数据集和特征

数据集是 Sklearn 最基本的概念之一。

机器学习的核心任务是从数据中学习模式，数据的表示方式至关重要。

数据集 (Dataset)

在 scikit-learn 中，数据通常通过两个主要的对象来表示：特征矩阵 和 目标向量。

特征矩阵 (Feature Matrix)：每一行代表一个数据样本，每一列代表一个特征（即输入变量）。它是一个二维的数组或矩阵，通常使用 NumPy 数组或 pandas DataFrame 来存储。

假设我们有 3 个样本，每个样本有 2 个特征。

```
1 import numpy as np
2 x = np.array([[1.0, 2.0], [2.0, 3.0], [3.0, 4.0]])
```

目标向量 (Target Vector)：它表示每个样本的目标（即输出标签），通常是一个一维数组。

例如，在分类任务中，目标是每个样本的类别标签。

对应的目标向量：

```
1 y = np.array([0, 1, 0]) # 0 类别和 1 类别
```

特征和标签

- **特征 (Features)**：是数据集中用于训练模型的输入变量。在上面的例子中，`x` 是特征矩阵，包含了所有的输入变量。
- **标签 (Labels)**：是机器学习模型的目标输出。在监督学习中，标签是我们希望模型预测的结果。在上面的例子中，`y` 是标签或目标向量，包含了每个样本的类别。

数据集分割

在实际应用中，通常需要将数据集分割成训练集和测试集。

scikit-learn 提供了一个方便的函数 `train_test_split()` 来实现这一点：

```
1 from sklearn.model_selection import train_test_split
2 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

- 以上代码调用 `train_test_split` 函数，并将结果赋值给四个变量：`x_train`、`x_test`、`y_train` 和 `y_test`。
- `x` 和 `y` 是传入 `train_test_split` 函数的参数，它们分别代表特征数据集和目标变量（标签）。通常 `x` 是一个二维数组，`y` 是一个一维数组。
- `test_size=0.3` 参数指定了测试集的大小应该是原始数据集的 30%。这意味着 70% 的数据将被用作训练集，剩下的 30% 将被用作测试集。

- `random_state=42` 参数是一个随机数种子，用于确保每次分割数据集时都能得到相同的结果。这在实验和模型验证中非常有用，因为它确保了结果的可重复性。

2、模型与算法

监督学习和无监督学习

在 scikit-learn 中，机器学习模型大致分为两大类：监督学习和无监督学习。

监督学习 (Supervised Learning)：在监督学习中，模型在训练时会利用带标签的数据进行学习，这些标签是我们希望模型预测的结果。

常见的监督学习任务包括分类和回归。

- **分类 (Classification)**：将数据点分配到预定的类别中。例如，判断邮件是垃圾邮件还是非垃圾邮件。
- **回归 (Regression)**：预测连续值输出。例如，预测房价、气温等。

使用决策树进行分类任务：

```
1 from sklearn.tree import DecisionTreeClassifier
2 clf = DecisionTreeClassifier()
3 clf.fit(X_train, y_train)
4 y_pred = clf.predict(X_test)
```

无监督学习 (Unsupervised Learning)：无监督学习是指没有标签数据，模型仅通过输入数据本身的特征进行学习。

常见的无监督学习任务包括聚类和降维。

- **聚类 (Clustering)**：将数据分组，使得同一组中的数据具有相似性。常见的聚类算法包括 **K-Means**、**DBSCAN** 等。
- **降维 (Dimensionality Reduction)**：减少数据中的特征数量，通常用于数据压缩或可视化。常见的降维方法有 **PCA**（主成分分析）和 **t-SNE**（t-分布随机邻域嵌入）等。

使用 K-Means 聚类：

```
1 from sklearn.cluster import KMeans
2 kmeans = KMeans(n_clusters=3)
3 kmeans.fit(X_train)
4 y_pred = kmeans.predict(X_test)
```

预处理与特征工程

在使用 scikit-learn 进行机器学习之前，通常需要对数据进行预处理，这包括以下几项常见任务：

1、标准化 (Standardization)：特征的尺度统一，使得每个特征都具有零均值和单位方差。

```
1 from sklearn.preprocessing import StandardScaler
2 scaler = StandardScaler()
3 X_scaled = scaler.fit_transform(X)
```

2、归一化 (Normalization)：将特征的值缩放到一个固定范围（通常是 0 到 1）。

```
1 from sklearn.preprocessing import MinMaxScaler
2 scaler = MinMaxScaler()
3 X_normalized = scaler.fit_transform(X)
```

3、类别变量编码：将类别型数据转换为数值型数据（如 one-hot 编码）。

3、模型评估与验证

在训练机器学习模型之后，必须评估其性能以确保其泛化能力。

scikit-learn 提供了一些评估模型性能的工具。

交叉验证 (Cross-validation)

交叉验证是一种常见的模型评估方法，尤其是在数据量有限的情况下。

通过将数据分成多个子集，每次使用一个子集作为验证集，其余作为训练集，重复多次训练和评估模型，最终计算模型的平均性能。

```
1 from sklearn.model_selection import cross_val_score
2 scores = cross_val_score(clf, X, y, cv=5) # 5-fold cross-validation
3 print("Cross-validation scores:", scores)
```

常见评估指标

分类任务的评估指标：

- **准确率 (Accuracy)**：预测正确的样本占所有样本的比例。
- **精确率 (Precision)**：正类预测中，实际正类的比例。
- **召回率 (Recall)**：实际正类中，正确预测的比例。
- **F1 分数**：精确率和召回率的调和平均数。

```
1 from sklearn.metrics import accuracy_score, classification_report
2 print("Accuracy:", accuracy_score(y_test, y_pred))
3 print("Classification Report:\n", classification_report(y_test, y_pred))
```

回归任务的评估指标：

- **均方误差 (MSE)**：预测值与真实值的平方差的平均值。
- **决定系数 (R^2)**：衡量模型对数据变异的解释能力。

```
1 from sklearn.metrics import mean_squared_error, r2_score
2 print("MSE:", mean_squared_error(y_test, y_pred))
3 print("R²:", r2_score(y_test, y_pred))
```

4、模型选择与调优

网格搜索 (Grid Search)

网格搜索是一种常用的超参数调优方法，它通过遍历所有可能的参数组合来寻找最佳的超参数组合。

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}
4 grid_search = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5)
5 grid_search.fit(X_train, y_train)
6 print("Best parameters:", grid_search.best_params_)
```

随机搜索 (Random Search)

随机搜索是一种通过随机选择超参数的组合来搜索最优超参数的方法，它比网格搜索效率高。

```
1 from sklearn.model_selection import RandomizedSearchCV
2 from scipy.stats import randint
3
4 param_dist = {'max_depth': [3, 5, 7], 'min_samples_split': randint(2, 10)}
5 random_search = RandomizedSearchCV(DecisionTreeClassifier(), param_dist,
6                                     n_iter=10, cv=5)
7 random_search.fit(X_train, y_train)
8 print("Best parameters:", random_search.best_params_)
```

数据预处理

数据预处理是机器学习项目中的一个关键步骤，它直接影响模型的训练效果和最终性能。

在进行机器学习建模时，数据预处理是至关重要的一步，它帮助我们清洗和转换原始数据，以便为机器学习模型提供最佳的输入。

数据预处理涉及多个步骤，包括处理缺失值、数据转换、标准化、编码等。

合适的预处理不仅能提高模型的准确性，还能帮助模型更好地泛化。

1、处理缺失值

缺失值是指在数据集中某些特征的值缺失。

机器学习算法通常无法直接处理缺失值，因此我们需要对缺失值进行处理。

检查缺失值

首先，检查数据集中是否有缺失值。

通常可以使用 pandas 来查看数据集中的缺失值：

```
1
2 import pandas as pd
3
4 # 假设我们有一个 DataFrame df
5 print(df.isnull().sum()) # 查看每一列缺失值的数量
```

填充缺失值

对于缺失值的处理，最常用的方法是填充。

常见的填充策略包括：

- **填充均值 (Mean)**：适用于数值型数据。
- **填充中位数 (Median)**：对于含有离群值的数据集，使用中位数可能更有效。
- **填充最频繁值 (Mode)**：适用于类别型数据。

在 scikit-learn 中，SimpleImputer 可以轻松实现缺失值填充：

```
1  
2 from sklearn.impute import SimpleImputer  
3  
4 # 对于数值型数据，使用均值填充  
5 imputer = SimpleImputer(strategy='mean') # 可选: 'mean', 'median',  
    'most_frequent'  
6 df_imputed = imputer.fit_transform(df) # 填充缺失值
```

删除缺失值

如果缺失值的数量较少，并且删除这些数据不会显著影响分析结果，另一种选择是直接删除缺失值。

```
1 df_cleaned = df.dropna() # 删除包含缺失值的行
```

更多内容可以参考：[Pandas 数据清洗](#)

2、数据缩放

机器学习算法对数据的尺度敏感，因此需要对数据进行缩放，使得特征具有相同的尺度。

常见的缩放方法有：

- **标准化 (Standardization)**：将数据转换为均值为0、标准差为1的分布。适用于大多数机器学习算法。
- **归一化 (Normalization)**：将数据缩放到指定范围（通常是 [0, 1]）。

标准化

标准化可以通过 StandardScaler 实现，它会将每个特征转换为零均值和单位方差：

```
1 from sklearn.preprocessing import StandardScaler  
2  
3 scaler = StandardScaler()  
4 X_scaled = scaler.fit_transform(X) # 标准化 X
```

归一化

归一化将每个特征缩放到一个指定的范围（通常是 [0, 1]）。

MinMaxScaler 用于将数据进行归一化：

```
1 from sklearn.preprocessing import MinMaxScaler
2
3 scaler = MinMaxScaler()
4 X_normalized = scaler.fit_transform(X) # 归一化 X
```

为什么需要标准化和归一化？

- **标准化**：对于距离度量（如 K 最近邻、支持向量机等）非常重要，因为特征的尺度不一致可能导致某些特征对模型的影响过大。标准化能确保每个特征对模型有相同的贡献。
- **归一化**：有些算法（如神经网络、梯度下降优化算法等）对输入数据的范围非常敏感，归一化有助于加速收敛。

3、类别变量编码

机器学习模型通常无法直接处理字符串类型的类别变量，因此需要将类别变量转化为数值型数据。

常见的编码方法有：

标签编码

标签编码将每个类别映射到一个唯一的整数。

适用于类别之间有顺序关系的情况（例如，低、中、高）。

```
1
2 from sklearn.preprocessing import LabelEncoder
3
4 # 假设我们有一个类别变量 y
5 label_encoder = LabelEncoder()
6 y_encoded = label_encoder.fit_transform(y) # 将类别变量转换为整数
```

独热编码

独热编码将每个类别转换为一个二进制的向量，适用于类别之间没有顺序关系的情况（例如，颜色、国家等）。

OneHotEncoder 可以将类别变量转化为独热编码。

```
1
2 from sklearn.preprocessing import OneHotEncoder
3
4 # 假设我们有一个类别变量 X
5 encoder = OneHotEncoder(sparse=False) # sparse=False 返回一个密集矩阵
6 X_encoded = encoder.fit_transform(X) # 将类别变量转换为独热编码
```

在 pandas 中，也可以使用 get_dummies() 函数进行独热编码：

```
1 X_encoded = pd.get_dummies(X)
```

4、特征选择

特征选择是通过选择最重要的特征来提高模型的性能，并减少计算成本。

常见的特征选择方法包括：

基于模型的特征选择

使用一些机器学习模型（如决策树或随机森林）来评估特征的重要性，从而进行特征选择。

```
1
2 from sklearn.ensemble import RandomForestClassifier
3
4 # 训练一个随机森林模型
5 clf = RandomForestClassifier()
6 clf.fit(X_train, y_train)
7
8 # 获取特征重要性
9 importances = clf.feature_importances_
10 print(importances)
```

递归特征消除（Recursive Feature Elimination, RFE）

RFE 是一种通过递归的方式，逐步删除最不重要的特征，从而选择最优特征的方法。

RFE 可以帮助我们自动选择重要特征。

```
1
2 from sklearn.feature_selection import RFE
3
4 # 使用线性模型进行递归特征消除
5 rfe = RFE(clf, n_features_to_select=3) # 保留 3 个最重要的特征
6 x_rfe = rfe.fit_transform(X_train, y_train)
```

5、特征工程

特征工程是通过对现有特征进行处理、组合或构造新的特征，以提高模型的表现。

特征工程的常见方式包括：

- **特征组合**：将两个或更多的特征组合成一个新特征。
- **特征转换**：对特征进行对数变换、平方根变换等，以解决数据的非线性问题。
- **特征创建**：根据现有数据创建新的特征，例如从时间戳中提取出日期、月份、星期等。

```
1
2 # 例如，将两个数值型特征组合成一个新特征
3 df['new_feature'] = df['feature1'] * df['feature2']
```

6、特征提取

特征提取旨在从原始特征中提取出新的、更具表达力的特征。

常见的特征提取方法包括: 主成分分析（PCA）和 线性判别分析（LDA）。

主成分分析 (PCA)

PCA 是一种常用的降维技术，它通过线性变换将数据从高维空间映射到低维空间，使得新特征（主成分）尽可能保留数据的方差。

PCA 特别适用于特征数量过多的情况，可以有效降低计算复杂度。

```
1
2 from sklearn.decomposition import PCA
3
4 # 假设 X 是特征矩阵
5 pca = PCA(n_components=2) # 降维到 2 个主成分
6 X_pca = pca.fit_transform(X)
```

PCA 主要用于两种场景：

- **降维**：当特征过多时，使用 PCA 降维可以减少计算成本，同时保留数据的主要信息。
- **可视化**：将高维数据映射到 2D 或 3D 空间，帮助我们可视化数据结构。

线性判别分析 (LDA)

LDA 是一种监督学习的降维方法，它旨在找到一个线性组合，使得不同类别之间的距离最大化，类别内的距离最小化。

LDA 通常用于分类任务中。

```
1
2 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
3
4 # 假设 X 是特征矩阵, y 是目标变量
5 lda = LinearDiscriminantAnalysis(n_components=2) # 降维到 2 个线性判别组件
6 X_lda = lda.fit_transform(X, y)
```

7、处理不平衡数据

在分类问题中，如果数据集的各类别样本数量差异较大，可能会导致模型偏向预测多数类，从而影响模型的性能。

常见的处理方法包括：

上采样 (Over-sampling)

通过增加少数类样本的数量，使得数据集更加平衡。

常见的方法是使用 SMOTE (Synthetic Minority Over-sampling Technique) 算法。

```
1
2 from imblearn.over_sampling import SMOTE
3
4 smote = SMOTE()
5 X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

下采样 (Under-sampling)

通过减少多数类样本的数量，使得数据集更加平衡。

```
1  
2 from imblearn.under_sampling import RandomUnderSampler  
3  
4 undersampler = RandomUnderSampler()  
5 x_resampled, y_resampled = undersampler.fit_resample(X_train, y_train)
```

Sklearn 机器学习模型

机器学习模型是实现自动化数据分析、模式识别和预测的核心工具。

根据任务类型的不同，机器学习模型通常可以分为：

- **分类模型 (Classification)**：预测离散类别
- **回归模型 (Regression)**：预测连续数值
- **聚类模型 (Clustering)**：对数据进行自动分组

本章节将详细介绍这些常见的机器学习模型，并介绍如何评估和优化模型。

1、分类模型

分类问题是机器学习中最常见的问题之一，其目的是将输入数据映射到离散的类别标签。

常见分类模型包括：

- 逻辑回归
- K-近邻 (KNN)
- 支持向量机 (SVM)
- 决策树
- 随机森林

逻辑回归 (Logistic Regression)

逻辑回归虽然名字中带"回归"，但本质是**概率型分类模型**，常用于二分类问题。

其核心思想是：

- 先进行线性加权求和
- 再通过 Sigmoid 函数将结果映射到 0~1 之间作为概率

形式可表示为：

逻辑回归的核心公式为：

$$P(y = 1|X) = \frac{1}{1 + e^{-(b_0 + b_1 X_1 + b_2 X_2 + \dots + b_n X_n)}}$$

其中， $X_1, X_2, \dots, X_n$ 为特征， $b_0, b_1, \dots, b_n$ 为回归系数， e 是自然对数的底。

scikit-learn 实现:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

假设 X 是特征矩阵， y 是标签

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

预测

```
y_pred = model.predict(X_test)
```

K-近邻 (K-Nearest Neighbors, KNN)

K-近邻 (KNN) 是一种基于实例的学习方法，预测时通过计算待预测样本与训练集中所有样本的距离，选取距离最近的 K 个邻居，并根据邻居的标签进行预测。

主要参数:

- **K**: 选择的邻居数量。
- **距离度量**: 常用欧氏距离，也可以使用曼哈顿距离、闵可夫斯基距离等。

scikit-learn 实现:

```
1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.model_selection import train_test_split
3
4 # 假设 x 是特征矩阵, y 是标签
5 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
6 random_state=42)
7
8 model = KNeighborsClassifier(n_neighbors=3)
9 model.fit(X_train, y_train)
10
11 # 预测
12 y_pred = model.predict(X_test)
```

支持向量机 (Support Vector Machine, SVM)

支持向量机是一种强大的分类模型，尤其适用于高维数据。

SVM 的基本思想是找到一个超平面，使得不同类别的样本点之间的间隔最大化。对于非线性可分的数据，SVM 通过核技巧将数据映射到高维空间，找到一个分隔超平面。

核函数:

- **线性核**: 适用于线性可分的数据。
- **高斯径向基核 (RBF)**: 适用于非线性数据。
- **多项式核**: 适用于具有多项式关系的数据。

scikit-learn 实现:

```
1
2 from sklearn.svm import SVC
3 from sklearn.model_selection import train_test_split
4
5 # 假设 X 是特征矩阵, y 是标签
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
7 random_state=42)
8
9 model = SVC(kernel='linear') # 使用线性核
10 model.fit(X_train, y_train)
11
12 # 预测
13 y_pred = model.predict(X_test)
```

决策树与随机森林 (Decision Tree & Random Forest)

决策树是一种树形结构的分类模型，通过对数据进行分裂，最终将数据划分到不同的类别。随机森林则是通过构建多棵决策树，并通过投票或平均来决定最终的预测结果。

决策树通过选择最优的特征进行数据划分，选择准则通常是 **信息增益** 或 **基尼系数**。

随机森林通过多棵决策树的集成来减少过拟合，并提高模型的准确性。它通过引入随机性（如随机选择特征、随机选择数据子集）来增加模型的多样性。

scikit-learn 实现:

```
1
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.model_selection import train_test_split
5
6 # 假设 X 是特征矩阵, y 是标签
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
8 random_state=42)
9
10 # 决策树
11 dt_model = DecisionTreeClassifier()
12 dt_model.fit(X_train, y_train)
13
14 # 随机森林
15 rf_model = RandomForestClassifier(n_estimators=100)
16 rf_model.fit(X_train, y_train)
17
18 # 预测
19 dt_pred = dt_model.predict(X_test)
20 rf_pred = rf_model.predict(X_test)
```

2、回归模型

回归问题的目标是预测一个连续的输出变量。常见的回归模型包括线性回归、岭回归和 Lasso 回归。

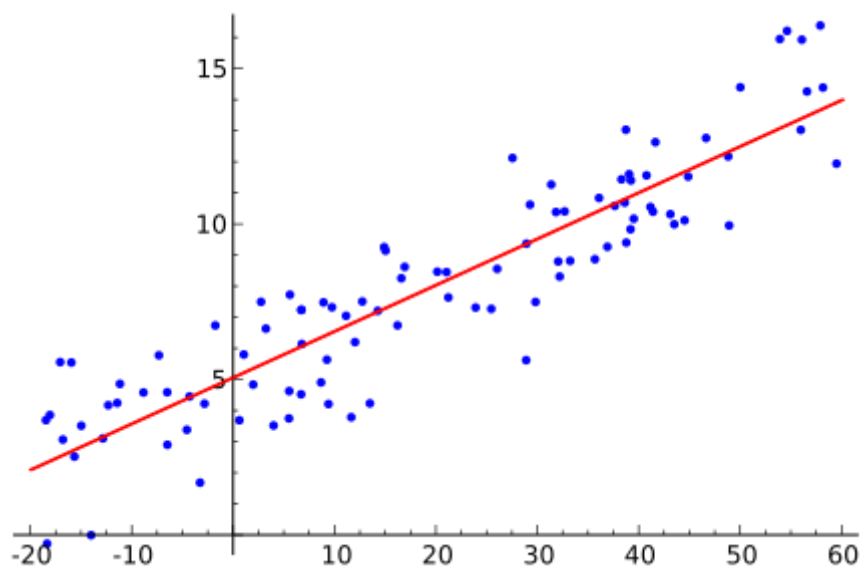
线性回归 (Linear Regression)

线性回归通过拟合一条直线来预测目标变量。其核心假设是特征与目标变量之间存在线性关系。

对于一个简单的线性回归问题，模型可以表示为：

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

- y 是预测值（目标值）。
- x_1, x_2, x_n 是输入特征。
- w_1, w_2, w_n 是待学习的权重（模型参数）。
- b 是偏置项。



scikit-learn 实现:

```
1
2 from sklearn.linear_model import LinearRegression
3 from sklearn.model_selection import train_test_split
4
5 # 假设 X 是特征矩阵, y 是目标变量
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
7 random_state=42)
8
9 model = LinearRegression()
10 model.fit(X_train, y_train)
11
12 # 预测
13 y_pred = model.predict(X_test)
```

岭回归 (Ridge Regression)

岭回归是线性回归的一个变种，使用 **L2 正则化** 来约束模型的复杂度，避免过拟合。通过惩罚回归系数的大小，岭回归能更好地处理多重共线性问题。

scikit-learn 实现:

```
1
2 from sklearn.linear_model import Ridge
3
4 model = Ridge(alpha=1.0) # alpha 是正则化参数
5 model.fit(X_train, y_train)
6
7 # 预测
8 y_pred = model.predict(X_test)
```

Lasso 回归 (Lasso Regression)

Lasso 回归也是线性回归的一种形式，它使用 **L1 正则化** 来对回归系数进行惩罚。与岭回归不同，Lasso 会将一些回归系数压缩到零，从而实现特征选择。

scikit-learn 实现:

```
1
2 from sklearn.linear_model import Lasso
3
4 model = Lasso(alpha=0.1) # alpha 是正则化参数
5 model.fit(X_train, y_train)
6
7 # 预测
8 y_pred = model.predict(X_test)
```

3、聚类模型

聚类是一种无监督学习方法，其目标是将数据集中的对象分为不同的组（或簇），使得同一簇中的对象尽可能相似，而不同簇之间的对象尽可能不同。

K-均值 (K-Means)

K-均值是一种常见的聚类算法，目标是将数据分为 K 个簇，通过最小化每个数据点与其簇中心的距离来优化簇划分。

scikit-learn 实现:

```
1
2 from sklearn.cluster import KMeans
3
4 # 假设 X 是特征矩阵
5 model = KMeans(n_clusters=3)
6 model.fit(X)
7
8 # 获取聚类标签
9 labels = model.predict(X)
```

DBSCAN (密度聚类)

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种基于密度的聚类算法，它通过寻找密度相对较高的区域来进行聚类，不需要事先指定簇的数量。

scikit-learn 实现:

```
1
2 from sklearn.cluster import DBSCAN
3
4 # 假设 X 是特征矩阵
5 model = DBSCAN(eps=0.5, min_samples=5)
6 model.fit(X)
7
8 # 获取聚类标签
9 labels = model.labels_
```

层次聚类 (Hierarchical Clustering)

层次聚类是一种通过递归地合并或分割簇的方式进行聚类的方法。常用的层次聚类方法有 **凝聚型聚类** (Agglomerative) 和 **分裂型聚类** (Divisive) 。

scikit-learn 实现:

```
1
2 from sklearn.cluster import AgglomerativeClustering
3
4 # 假设 X 是特征矩阵
5 model = AgglomerativeClustering(n_clusters=3)
6 labels = model.fit_predict(X)
```

4、模型评估与选择

在机器学习中，模型训练完成后，需要通过合理的评估方法来判断模型的泛化能力，并选择性能最优的模型。

常见评估方式包括：

- 分类指标评估
- 交叉验证
- 超参数调优

分类评估指标

在分类问题中，常用的评价指标包括：

- **准确率 (Accuracy)**
- **精确率 (Precision)**
- **召回率 (Recall)**
- **F1 分数 (F1 Score)**

它们从不同角度反映模型性能。

各指标含义说明：

- **准确率 (Accuracy)** -- 所有样本中预测正确的比例：适合类别分布较均衡的情况
- **精确率 (Precision)** -- 预测为正类的样本中，真正为正的比例：关注"预测为正是否靠谱"
- **召回率 (Recall)** -- 所有实际为正类的样本中，被模型正确识别出来的比例：关注"正样本是否被漏掉"

F1 分数

Precision 与 Recall 的调和平均值:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

用于在精确率和召回率之间取得平衡。

scikit-learn 实现:

```
1
2 from sklearn.metrics import accuracy_score, precision_score, recall_score,
  f1_score
3
4 accuracy = accuracy_score(y_test, y_pred)
5 precision = precision_score(y_test, y_pred)
6 recall = recall_score(y_test, y_pred)
7 f1 = f1_score(y_test, y_pred)
8
9 print(f"Accuracy: {accuracy}")
10 print(f"Precision: {precision}")
11 print(f"Recall: {recall}")
12 print(f"F1 Score: {f1}")
```

留出法评估（训练集 / 测试集划分）

最常见、最直观的评估方式是将数据集划分为：

- 训练集：用于模型学习
- 测试集：用于性能评估

该方法简单高效，适合入门阶段理解模型效果。

交叉验证（Cross Validation）

交叉验证通过多次划分数据集进行训练与测试，从而获得更加稳定、可靠的模型性能评估结果。

常见做法是 **K 折交叉验证**：

- 数据集被划分为 K 份
- 每次用其中一份作为验证集
- 其余作为训练集
- 重复 K 次取平均结果

```
1
2 from sklearn.model_selection import cross_val_score
3 from sklearn.linear_model import LogisticRegression
4
5 model = LogisticRegression()
6
7 scores = cross_val_score(model, X, y, cv=5)
8
9 print("Cross-validation scores:", scores)
10 print("Mean score:", scores.mean())
```

实际项目中通常要么使用留出法，要么使用交叉验证，不需要同时混用。

网格搜索调参 (GridSearchCV)

模型性能往往受到超参数设置影响较大。

网格搜索通过遍历参数组合，自动寻找最优配置。

同时结合交叉验证，使调参结果更加稳健。

```
1
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.svm import SVC
4
5 parameters = {
6     'kernel': ['linear', 'rbf'],
7     'C': [1, 10, 100]
8 }
9
10 model = SVC()
11
12 grid_search = GridSearchCV(
13     model,
14     parameters,
15     cv=5
16 )
17
18 grid_search.fit(X, y)
19
20 print("Best parameters:", grid_search.best_params_)
21 print("Best score:", grid_search.best_score_)
```

Sklearn 模型评估与调优

机器学习模型的评估与调优是确保模型泛化能力和预测准确度的关键步骤。

通过合适的评估指标和调优方法，可以有效提高模型性能，避免过拟合或欠拟合的风险。

本章节将详细介绍交叉验证、网格搜索、随机搜索、模型评估方法等内容。

1、交叉验证

介绍交叉验证的概念

交叉验证 (Cross-Validation) 是一种用于评估模型性能的技术，它通过将数据集分成多个子集（折叠），并多次训练和测试模型来获得更稳定、可靠的评估结果。

交叉验证有助于检测模型是否过拟合，并且能够更准确地评估模型的泛化能力。

常见的交叉验证方法包括：

- **K-fold 交叉验证**：将数据分成 K 个折叠，依次选择其中一个折叠作为测试集，其他 K-1 个折叠作为训练集，重复 K 次，最后计算 K 次的结果平均值。
- **留一法交叉验证 (Leave-One-Out Cross-Validation, LOOCV)**：每次只保留一个数据点作为测试集，剩余的作为训练集。这种方法非常耗时，但可以用于小数据集。
- **分层 K-fold 交叉验证**：在 K-fold 中，确保每个折叠中的类别分布与整个数据集相似，适用于类别不平衡的情况。

scikit-learn 提供了多种交叉验证的方法，如 `cross_val_score` 和 `cross_val_predict` 等，可以帮助我们高效地进行交叉验证。

使用 `cross_val_score` 执行 K-fold 交叉验证

`cross_val_score` 函数用于执行 K-fold 交叉验证，返回每个折叠的评分结果，帮助我们评估模型的稳定性和性能。

使用 `cross_val_score` 执行 K-fold 交叉验证:

```
1
2 from sklearn.model_selection import cross_val_score
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.datasets import load_iris
5
6 # 加载数据
7 data = load_iris()
8 X, y = data.data, data.target
9
10 # 创建模型
11 model = RandomForestClassifier()
12
13 # 执行 k-fold 交叉验证
14 scores = cross_val_score(model, X, y, cv=5) # 5-fold 交叉验证
15 print(f"Cross-validation scores: {scores}")
16 print(f"Mean accuracy: {scores.mean()}")
```

- `cv=5`: 表示进行 5 折交叉验证。
- `scores`: 返回每一折的评分，最终结果是这些评分的平均值，表示模型的性能。

输出如下所示:

```
1 Cross-validation scores: [0.96666667 0.96666667 0.93333333 0.96666667 1.
2 ]
3 Mean accuracy: 0.9666666666666668
```

2、网格搜索与随机搜索

使用 `GridSearchCV` 进行超参数调优

`GridSearchCV` 是一种通过穷举搜索所有超参数组合来找到最佳超参数的技术。

`GridSearchCV` 通过提供一组参数的候选值，计算每一种组合的性能，最终选择最佳的参数组合。

GridSearchCV 的常见参数:

- `param_grid`: 待调优的超参数网格，通常是一个字典，键是参数名，值是参数的候选值。
- `cv`: 交叉验证的折数，通常设置为 5 或 10。

scikit-learn 使用 `GridSearchCV` 实例:

```
1
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.svm import SVC
4 from sklearn.datasets import load_iris
```

```

5
6 # 加载数据
7 data = load_iris()
8 X, y = data.data, data.target
9
10 # 创建模型
11 model = SVC()
12
13 # 定义超参数网格
14 param_grid = {'kernel': ['linear', 'rbf'], 'C': [1, 10, 100]}
15
16 # 执行网格搜索
17 grid_search = GridSearchCV(model, param_grid, cv=5)
18 grid_search.fit(X, y)
19
20 # 输出最佳参数和最佳得分
21 print(f"Best parameters: {grid_search.best_params_}")
22 print(f"Best score: {grid_search.best_score_}")

```

- `grid_search.best_params_`：返回网格搜索中表现最好的超参数组合。
- `grid_search.best_score_`：返回最佳参数组合下的交叉验证得分。

输出如下所示：

```

1 Best parameters: {'C': 1, 'kernel': 'linear'}
2 Best score: 0.9800000000000001

```

使用 RandomizedSearchCV 加速调优过程

RandomizedSearchCV 是一种更高效的超参数调优方法，它通过从超参数空间中随机选择一定数量的组合进行评估，从而加速调优过程。

RandomizedSearchCV 适用于超参数空间较大时，可以节省计算时间。

RandomizedSearchCV 的常见参数：

- `param_distributions`：待调优的超参数分布，通常是一个字典，值可以是分布对象（如 `scipy.stats` 中的分布）或者离散的值列表。
- `n_iter`：随机搜索的迭代次数，即随机选择的超参数组合数量。

scikit-learn 使用 RandomizedSearchCV 实例：

```

1
2 from sklearn.model_selection import RandomizedSearchCV
3 from sklearn.svm import SVC
4 from sklearn.datasets import load_iris
5 from scipy.stats import uniform
6
7 # 加载数据
8 data = load_iris()
9 X, y = data.data, data.target
10
11 # 创建模型
12 model = SVC()
13
14 # 定义超参数分布
15 param_distributions = {'C': uniform(0, 10), 'kernel': ['linear', 'rbf']}

```

```

16
17 # 执行随机搜索
18 random_search = RandomizedSearchCV(model, param_distributions, n_iter=10,
19 cv=5)
20
21 random_search.fit(X, y)
22
23 # 输出最佳参数和最佳得分
24 print(f"Best parameters: {random_search.best_params_}")
25 print(f"Best score: {random_search.best_score_}")

```

- `random_search.best_params_`：返回随机搜索中表现最好的超参数组合。
- `random_search.best_score_`：返回最佳参数组合下的交叉验证得分。

输出如下所示：

```

1 Best parameters: {'C': 8.355688344706016, 'kernel': 'rbf'}
2 Best score: 0.9866666666666667

```

3、评估模型

使用 `classification_report`, `confusion_matrix`, `roc_auc_score`

对于分类模型，我们通常使用精度、召回率、F1 分数等指标来评估模型性能。

scikit-learn 提供了许多评估工具，帮助我们深入了解模型的表现。

`classification_report` - 提供了精度、召回率、F1 分数和支持度（每个类别的样本数）等信息。

```

1
2 from sklearn.metrics import classification_report
3
4 # 假设 y_test 是真实标签, y_pred 是模型预测结果
5 print(classification_report(y_test, y_pred))

```

`confusion_matrix` - 混淆矩阵用于显示分类模型在各个类别上的表现，特别是如何将正类预测为负类，反之亦然。

```

1
2 from sklearn.metrics import confusion_matrix
3
4 # 假设 y_test 是真实标签, y_pred 是模型预测结果
5 print(confusion_matrix(y_test, y_pred))

```

`roc_auc_score` - ROC AUC（接收者操作特征曲线下面积）是评估分类模型性能的指标，尤其适用于不平衡数据集。AUC 值越高，模型性能越好。

```

1
2 from sklearn.metrics import roc_auc_score
3
4 # 假设 y_test 是真实标签, y_pred_proba 是模型预测的概率值
5 print(f"ROC AUC Score: {roc_auc_score(y_test, y_pred_proba)}")

```

回归模型评估: mean_squared_error, r2_score

对于回归问题，常用的评估指标包括 均方误差 (MSE) 和 决定系数 (R^2)。

mean_squared_error - 均方误差是回归模型的常见评估标准，计算预测值与真实值之间的平方误差的均值。

```
1 from sklearn.metrics import mean_squared_error
2
3 # 假设 y_test 是真实值, y_pred 是预测值
4 print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred)}")
```

r2_score - 决定系数 R^2 用于衡量模型对数据的拟合程度，值越接近 1，表示模型拟合得越好。

```
1
2 from sklearn.metrics import r2_score
3
4 # 假设 y_test 是真实值, y_pred 是预测值
5 print(f"R² Score: {r2_score(y_test, y_pred)}")
```

Sklearn 管道 (Pipeline)

在机器学习项目中，数据处理、特征工程、模型训练、评估等步骤往往是互相依赖的，这些步骤的顺序和协调性对于最终模型的性能至关重要。

Pipeline 是 scikit-learn 中用于组织和简化这些步骤的一个重要工具。

通过 Pipeline，我们可以将数据预处理与模型训练整合在一起，从而简化工作流并提高代码的可复用性。

什么是 Pipeline

Pipeline 是一个可按顺序执行多个数据处理步骤和模型训练步骤的工具。

在 Pipeline 中，每个步骤是一个元组，包含一个名称和一个对象。

每个对象通常是一个 转换器 (Transformer) 或 估计器 (Estimator)，其中：

- **转换器 (Transformer)** 是执行数据转换的对象，比如数据预处理（例如归一化、标准化、特征选择等）。
- **估计器 (Estimator)** 是用于训练模型的对象，例如分类器或回归器。

`Pipeline` 使得将多个步骤整合为一个可重用的工作流变得简单，并且可以确保数据处理过程的一致性，避免因代码重复或手动处理导致的错误。

为什么要使用 Pipeline

- **简化代码**：将多个步骤组合成一个整体，简化了代码结构和管理。
- **避免数据泄漏**：在数据预处理时，确保训练集和测试集的处理是隔离的，避免数据泄漏。比如，标准化操作时，不能在测试集上计算均值和标准差。
- **减少重复工作**：通过 `Pipeline` 可以把数据预处理与模型训练过程串联起来，避免在每次训练时重复写预处理代码。
- **提高可复用性**：将数据处理和模型训练封装成一个 `Pipeline` 对象，可以在不同的项目和数据集上复用。
- **方便调优**：通过 `Pipeline` 可以直接在调优时应用超参数优化、交叉验证等，简化整个流程。

Pipeline 的组成部分

Pipeline 由多个步骤 (step) 组成, 每个步骤是一个元组, 包含两个元素:

1. **步骤名称** (字符串类型): 用于标识每个步骤。
2. **转换器或估计器**: 用于数据处理或建模的对象。

常见的步骤包括:

- **数据预处理步骤**: 如数据清洗、标准化、编码等。
- **模型训练步骤**: 如分类器、回归器等。

创建一个简单的 Pipeline

假设我们有一个数据集, 并且需要对数据进行标准化后训练一个支持向量机 (SVM) 分类器。我们可以将标准化和模型训练步骤组合成一个管道 (Pipeline)。

```
1
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.svm import SVC
5 from sklearn.model_selection import train_test_split
6 from sklearn.datasets import load_iris
7
8 # 加载数据
9 data = load_iris()
10 X, y = data.data, data.target
11
12 # 将数据集划分为训练集和测试集
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
14                                                    random_state=42)
15
16 # 创建 Pipeline
17 pipeline = Pipeline([
18     ('scaler', StandardScaler()), # 数据标准化
19     ('svc', SVC()) # 支持向量机分类器
20 ])
21
22 # 训练模型
23 pipeline.fit(X_train, y_train)
24
25 # 预测结果
26 y_pred = pipeline.predict(X_test)
27
28 # 打印模型精度
29 print(f"Model accuracy: {pipeline.score(X_test, y_test)}")
```

执行以上代码, 输出如下:

```
1 Model accuracy: 1.0
```

Pipeline 工作原理

在上面的示例中，Pipeline 执行了两个步骤：

1. **数据标准化**（通过 `StandardScaler()`）：对数据进行标准化处理，使每个特征具有均值 0 和方差 1。
2. **模型训练**（通过 `SVC()`）：在标准化后的数据上训练支持向量机分类器。

Pipeline 的工作流程是：首先执行数据预处理步骤（如标准化），然后传递处理后的数据给模型进行训练。这个过程可以通过 `pipeline.fit()` 一步完成，`pipeline.predict()` 进行预测时，数据也会按照相同的顺序通过管道中的每个步骤。

Pipeline 的优势

简化代码和流程

通过 Pipeline，我们可以将多个步骤整合成一个对象，从而减少了手动执行多个步骤的代码。

没有使用，需要多次执行预处理：

```
1
2 # without Pipeline (需要多次执行预处理)
3 scaler = StandardScaler()
4 X_train_scaled = scaler.fit_transform(X_train)
5 X_test_scaled = scaler.transform(X_test)
6
7 model = SVC()
8 model.fit(X_train_scaled, y_train)
9 y_pred = model.predict(X_test_scaled)
```

使用 Pipeline，一步完成：

```
1
2 # with Pipeline (一步完成)
3 pipeline = Pipeline([
4     ('scaler', StandardScaler()),
5     ('svc', SVC())
6 ])
7
8 pipeline.fit(X_train, y_train)
9 y_pred = pipeline.predict(X_test)
```

保证训练和测试数据处理一致性

在没有 Pipeline 时，如果我们手动进行数据处理和训练，可能会不小心对训练集和测试集使用不同的处理方法，导致数据泄漏。

例如，我们在训练集上计算标准化的均值和方差，但如果在测试集上计算了不同的均值和方差，就会导致模型的评估不准确。使用 Pipeline 可以确保这些处理方法的一致性。

自动化整个过程

Pipeline 可以让我们将多个步骤封装成一个对象，自动化整个数据预处理、模型训练和预测的过程。通过这个自动化的流程，可以减少人为错误，并提高代码的可复用性。

Pipeline 的调参与优化

当我们使用 Pipeline 时，可以直接进行超参数调优。

通过结合 GridSearchCV 或 RandomizedSearchCV，可以优化管道中的每一个步骤的超参数。

使用 GridSearchCV 调优 Pipeline 中的超参数:

```
1  from sklearn.pipeline import Pipeline
2  from sklearn.preprocessing import StandardScaler
3  from sklearn.svm import SVC
4  from sklearn.model_selection import train_test_split
5  from sklearn.datasets import load_iris
6  from sklearn.model_selection import GridSearchCV
7
8
9  # 加载数据
10 data = load_iris()
11 X, y = data.data, data.target
12
13 # 将数据集划分为训练集和测试集
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
15                                                    random_state=42)
16
17 # 创建 Pipeline
18 pipeline = Pipeline([
19     ('scaler', StandardScaler()), # 数据标准化
20     ('svc', SVC()) # 支持向量机分类器
21 ])
22
23 # 训练模型
24 pipeline.fit(X_train, y_train)
25
26 # 定义超参数网格
27 param_grid = {
28     'svc__C': [0.1, 1, 10], # 调整 SVC 中的 C 参数
29     'svc__kernel': ['linear', 'rbf'] # 调整 kernel 参数
30 }
31
32 # 创建 GridSearchCV 对象
33 grid_search = GridSearchCV(pipeline, param_grid, cv=5)
34
35 # 执行超参数调优
36 grid_search.fit(X_train, y_train)
37
38 # 输出最佳参数和得分
39 print(f"Best parameters: {grid_search.best_params_}")
40 print(f"Best score: {grid_search.best_score_}")
```

说明:

- `svc__C` 和 `svc__kernel` 是 `Pipeline` 中 `SVC` 步骤的超参数。通过在 `GridSearchCV` 中指定这些参数，我们可以直接对 `Pipeline` 中的模型进行超参数调优。
- `cv=5` 表示 5 折交叉验证。

输出结果如下：

```
1 Best parameters: {'svc__C': 0.1, 'svc__kernel': 'linear'}
2 Best score: 0.9583333333333334
```

使用 Pipeline 进行交叉验证

可以结合 `Pipeline` 和交叉验证，确保整个模型的评估过程是一致的。在交叉验证中，每次迭代都会对训练数据进行预处理，然后训练模型进行验证。

```
1
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.svm import SVC
5 from sklearn.model_selection import train_test_split
6 from sklearn.datasets import load_iris
7 from sklearn.model_selection import cross_val_score
8
9 # 加载数据
10 data = load_iris()
11 X, y = data.data, data.target
12
13 # 将数据集划分为训练集和测试集
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
15                                                    random_state=42)
16
17 # 创建 Pipeline
18 pipeline = Pipeline([
19     ('scaler', StandardScaler()), # 数据标准化
20     ('svc', SVC()) # 支持向量机分类器
21 ])
22
23 # 训练模型
24 pipeline.fit(X_train, y_train)
25
26 # 执行 5 折交叉验证
27 cv_scores = cross_val_score(pipeline, X, y, cv=5)
28
29 # 输出交叉验证分数
30 print(f"Cross-validation scores: {cv_scores}")
31 print(f"Mean cross-validation score: {cv_scores.mean()}")
```

在这个例子中，`cross_val_score` 会自动对数据进行交叉验证，同时在每次训练前对数据进行标准化处理。

输出结果如下：

```
1 Cross-validation scores: [0.96666667 0.96666667 0.96666667 0.93333333 1.
2                             ]
3 Mean cross-validation score: 0.9666666666666666
```

Sklearn 自定义模型与功能

自定义模型可以扩展 scikit-learn 的功能，以便满足特定应用场景下的需求。

在 scikit-learn 中，除了使用现成的模型和预处理功能外，用户还可以根据自己的需求创建自定义的模型、转换器和功能。

自定义模型和功能的实现通常涉及继承 scikit-learn 的基类，如 `BaseEstimator` 和 `TransformerMixin`，然后实现特定的 `fit` 和 `predict` 方法。

本章节会在以下几个方面展开说明：

1. 自定义转换器 (Transformer)
2. 自定义估计器 (Estimator)
3. 自定义管道步骤
4. 如何通过继承 `BaseEstimator` 和 `TransformerMixin` 实现自定义算法

1、自定义转换器 (Transformer)

转换器是用于对数据进行转换的组件，例如标准化、特征选择等。自定义转换器可以继承 `TransformerMixin`，并实现 `fit` 和 `transform` 方法。

自定义转换器的步骤：

- **fit 方法**：通常用于学习数据的属性（如均值、方差、特征选择标准等）。`fit` 方法返回转换器本身，以便可以进行链式调用。
- **transform 方法**：应用已学习的属性，对数据进行转换或处理。

自定义转换器示例：自定义标准化转换器：

假设我们希望实现一个自定义的标准化转换器，标准化是将数据的每个特征缩放到均值为 0、方差为 1 的范围。

```
1 import numpy as np
2 from sklearn.base import BaseEstimator, TransformerMixin
3
4 class CustomScaler(BaseEstimator, TransformerMixin):
5     def fit(self, X, y=None):
6         """
7         计算每个特征的均值和标准差
8         """
9         self.mean_ = np.mean(X, axis=0)
10        self.std_ = np.std(X, axis=0)
11        return self # 返回对象本身
12
13    def transform(self, X):
14        """
15        标准化数据
16        """
17        return (X - self.mean_) / self.std_
18
19 # 测试自定义转换器
20 from sklearn.datasets import load_iris
21 from sklearn.model_selection import train_test_split
22
23 # 加载数据
24 data = load_iris()
25 X, y = data.data, data.target
```

```

26 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
27 random_state=42)
28 # 创建自定义转换器对象
29 scaler = CustomScaler()
30
31 # 使用自定义标准化
32 scaler.fit(X_train)
33 X_train_scaled = scaler.transform(X_train)
34 X_test_scaled = scaler.transform(X_test)
35
36 print("Scaled training data:\n", X_train_scaled)

```

输出如下：

```

1 Scaled training data:
2 [[-1.47393679  1.20365799 -1.56253475 -1.31260282]
3 [-0.13307079  2.99237573 -1.27600637 -1.04563275]
4 [ 1.08589829  0.08570939  0.38585821  0.28921757]
5 [-1.23014297  0.75647855 -1.2187007  -1.31260282]
6 [-1.7177306   0.30929911 -1.39061772 -1.31260282]
7 ....

```

说明：

CustomScaler 类实现了一个标准化过程，类似于 scikit-learn 中的 StandardScaler。

- **fit** 方法计算训练数据的均值和标准差，并保存这些值。
- **transform** 方法根据 **fit** 中计算的均值和标准差来转换数据。

2、自定义估计器 (Estimator)

估计器是指模型本身，如回归器、分类器等。

自定义估计器需要继承 BaseEstimator 类，并实现 fit 和 predict 方法。

自定义估计器的步骤：

- **fit 方法**：用于训练模型，计算需要的参数（如权重、偏置等）。
- **predict 方法**：基于训练好的参数对输入数据进行预测。

自定义估计器示例：简单的分类器

假设我们要实现一个非常简单的分类器：将每个特征的均值作为阈值，超过均值的预测为类别 1，否则预测为类别 0。

```

1 from sklearn.base import BaseEstimator
2 import numpy as np
3
4 class SimpleClassifier(BaseEstimator):
5     def fit(self, X, y):
6         """
7         训练模型：计算每个特征的均值
8         """
9         self.mean_ = np.mean(X, axis=0)
10        return self # 返回对象本身

```

```

11
12     def predict(self, x):
13         """
14         基于均值进行分类：如果特征值大于均值，则预测为 1，否则为 0
15         """
16         return (X > self.mean_).astype(int)
17
18 # 测试自定义分类器
19 X_train = np.array([[1.5, 2.5], [2.0, 3.0], [3.5, 4.5], [4.0, 5.0]])
20 y_train = np.array([0, 0, 1, 1])
21
22 # 创建自定义分类器对象
23 classifier = SimpleClassifier()
24
25 # 训练模型
26 classifier.fit(X_train, y_train)
27
28 # 进行预测
29 X_test = np.array([[2.5, 3.5], [1.0, 2.0]])
30 y_pred = classifier.predict(X_test)
31
32 print("Predictions:", y_pred)

```

输出如下：

```

1 Predictions: [[0 0]
2               [0 0]]

```

说明：

- `fit` 方法计算训练数据的均值并将其存储在 `self.mean_` 中。
- `predict` 方法通过比较测试数据与均值的大小，做出分类预测。

这种自定义的分类器在实际中并不常用，但它展示了如何使用 `BaseEstimator` 创建一个简单的模型。

3、自定义管道步骤

scikit-learn 允许将自定义转换器和估计器作为管道的步骤。这使得可以将数据预处理和模型训练过程整合成一个工作流。自定义模型和功能可以像内建的转换器和估计器一样，与管道一起工作。

使用自定义转换器和估计器在管道中

```

1 import numpy as np
2 from sklearn.base import BaseEstimator, TransformerMixin
3 from sklearn.pipeline import Pipeline
4
5 class CustomScaler(BaseEstimator, TransformerMixin):
6     def fit(self, X, y=None):
7         """
8         计算每个特征的均值和标准差
9         """
10        self.mean_ = np.mean(X, axis=0)
11        self.std_ = np.std(X, axis=0)
12        return self # 返回对象本身
13

```

```

14     def transform(self, X):
15         """
16         标准化数据
17         """
18         return (X - self.mean_) / self.std_
19
20     class SimpleClassifier(BaseEstimator):
21     def fit(self, X, y):
22         """
23         训练模型：计算每个特征的均值
24         """
25         self.mean_ = np.mean(X, axis=0)
26         return self # 返回对象本身
27
28     def predict(self, X):
29         """
30         基于均值进行分类：如果特征值大于均值，则预测为 1，否则为 0
31         """
32         return (X > self.mean_).astype(int)
33
34 # 测试自定义分类器
35 X_train = np.array([[1.5, 2.5], [2.0, 3.0], [3.5, 4.5], [4.0, 5.0]])
36 y_train = np.array([0, 0, 1, 1])
37
38 # 创建管道，包含自定义的标准化和分类器
39 pipeline = Pipeline([
40     ('scaler', CustomScaler()), # 自定义标准化
41     ('classifier', SimpleClassifier()) # 自定义分类器
42 ])
43
44 # 训练管道
45 pipeline.fit(X_train, y_train)
46
47 X_test = np.array([[2.5, 3.5], [1.0, 2.0]])
48 # 预测
49 y_pred = pipeline.predict(X_test)
50 print("Predictions:", y_pred)

```

输出如下：

```

1 Predictions: [[0 0]
2               [0 0]]

```

说明：

我们将 CustomScaler 和 SimpleClassifier 作为管道的步骤，利用管道自动执行数据预处理和模型训练。

这样，管道的整个工作流可以通过一个 fit 和 predict 方法完成，保持了高效性和简洁性。

如何通过继承 BaseEstimator 和 TransformerMixin 实现自定义算法

- **BaseEstimator**：是所有 scikit-learn 估计器的基类。它提供了 `get_params()` 和 `set_params()` 方法，这使得自定义估计器能够与 `GridSearchCV` 等工具配合工作。

- **TransformerMixin**：是所有 scikit-learn 转换器的基类，它提供了 `fit_transform()` 方法，使得转换器能够与 **Pipeline** 一起使用。

通过继承这两个基类，我们可以非常方便地创建自己的自定义模型和功能，并且可以利用 scikit-learn 的工具（如 GridSearchCV 和 Pipeline）进行调优和评估。

完整自定义转换器和估计器示例:

```
1
2 from sklearn.base import BaseEstimator, TransformerMixin
3 import numpy as np
4
5 class CustomEstimator(BaseEstimator, TransformerMixin):
6     def fit(self, X, y=None):
7         # 模拟一个简单的"模型": 计算每个特征的均值
8         self.mean_ = np.mean(X, axis=0)
9         return self
10
11     def transform(self, X):
12         # 基于均值将数据标准化
13         return X - self.mean_
14
15     def predict(self, X):
16         # 简单的预测方法: 如果特征值大于均值, 则预测为 1, 否则为 0
17         return (X > self.mean_).astype(int)
18
19 # 使用自定义估计器和转换器
20 from sklearn.pipeline import Pipeline
21 from sklearn.datasets import load_iris
22 from sklearn.model_selection import train_test_split
23
24 # 加载数据
25 data = load_iris()
26 X, y = data.data, data.target
27 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
28                                                     random_state=42)
29
30 # 创建管道
31 pipeline = Pipeline([
32     ('custom', CustomEstimator())
33 ])
34
35 # 训练模型
36 pipeline.fit(X_train, y_train)
37
38 # 预测
39 y_pred = pipeline.predict(X_test)
40 print("Predictions:", y_pred)
```

以上例子中，我们创建了一个自定义的估计器 CustomEstimator，它不仅执行标准化（transform），还执行预测（predict）。然后，我们将其作为管道的一部分进行训练和预测。

输出如下所示：

```
1 Predictions: [[1 0 1 1]
2 [0 1 0 0]
3 [1 0 1 1]
4 [1 0 1 1]
5 [1 0 1 1]
6 [0 1 0 0]
7 [0 0 0 1]
8 [1 1 1 1]
9 [1 0 1 1]
10 [0 0 1 1]
11 ...
```

Sklearn 模型保存与加载

在机器学习中，模型的训练过程通常是耗时的，为了避免每次重新训练模型，我们可以将训练好的模型保存下来，便于以后进行加载和预测。

`scikit-learn` 提供了两种常用的方式来保存和加载模型：`joblib` 和 `pickle`。

1、使用 `joblib` 保存与加载模型

`joblib` 是一个高效的 Python 序列化工具，特别适合用于保存包含大量数值数组（如 `numpy` 数组、`scikit-learn` 模型等）的对象。相较于 `pickle`，`joblib` 在处理大规模数据时更高效。

`joblib` 是 Python 的一个外部库，可以通过以下命令安装：

```
1 pip install joblib
```

保存模型

`joblib` 提供了一个简单的 API 来保存和加载对象。

我们可以使用 `joblib.dump()` 方法将模型保存到文件中。

```
1 import joblib
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.svm import SVC
5
6 # 加载数据
7 data = load_iris()
8 X, y = data.data, data.target
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
12 random_state=42)
13
14 # 创建并训练模型
15 model = SVC(kernel='linear')
16 model.fit(X_train, y_train)
17
18 # 保存模型到文件
19 joblib.dump(model, 'svm_model.joblib')
```

加载模型

使用 `joblib.load()` 方法加载保存的模型对象。

```
1 # 加载保存的模型
2 loaded_model = joblib.load('svm_model.joblib')
3
4 # 使用加载的模型进行预测
5 y_pred = loaded_model.predict(X_test)
6
7 # 打印预测结果
8 print("Predictions:", y_pred)
```

通过上述步骤，我们成功地将训练好的模型保存到文件中，并在之后的任何时间加载该模型并进行预测。

2、使用 pickle 保存与加载模型

`pickle` 是 Python 内置的模块，允许将 Python 对象序列化和反序列化。

虽然 `joblib` 更适用于处理大量数据，但 `pickle` 也是常用的保存和加载模型的工具，适用于一般情况。

保存模型

与 `joblib` 类似，`pickle` 也有简单的 API 来保存和加载对象。

保存模型的代码如下：

```
1 import pickle
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.svm import SVC
5
6 # 加载数据
7 data = load_iris()
8 X, y = data.data, data.target
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
12                                                    random_state=42)
13
14 # 创建并训练模型
15 model = SVC(kernel='linear')
16 model.fit(X_train, y_train)
17
18 # 使用 pickle 保存模型
19 with open('svm_model.pkl', 'wb') as f:
20     pickle.dump(model, f)
```

加载模型

使用 `pickle.load()` 加载模型:

```
1 # 使用 pickle 加载保存的模型
2 with open('svm_model.pkl', 'rb') as f:
3     loaded_model = pickle.load(f)
4
5 # 使用加载的模型进行预测
6 y_pred = loaded_model.predict(X_test)
7
8 # 打印预测结果
9 print("Predictions:", y_pred)
```

3、joblib vs pickle

joblib 和 pickle 是保存和加载模型的两种常用方法。

joblib 更适合保存大型数据对象，而 pickle 是 Python 的标准序列化工具，适用于一般情况。

- **joblib**: 通常适用于保存包含大量数值数据（如 numpy 数组）的对象。joblib 在处理大规模数据时比 pickle 更高效。
- **pickle**: 适用于保存较小的对象或常规的 Python 对象。它是 Python 的内置库，使用时无需额外安装。

如果模型中包含大量数值数组或矩阵（如支持向量机、随机森林等），推荐使用 joblib，它比 pickle 更高效。对于较小的模型或不包含大量数值数据的模型，pickle 足够使用。

4、保存和加载管道 (Pipeline)

在实际应用中，模型不仅仅是单一的模型，有时会结合多个处理步骤（如数据预处理、特征选择、模型训练等），这些处理步骤可以使用 scikit-learn 的 Pipeline 来完成。Pipeline 也可以通过 joblib 或 pickle 保存和加载。

保存管道:

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.svm import SVC
4 import joblib
5
6 # 创建一个管道
7 pipeline = Pipeline([
8     ('scaler', StandardScaler()),
9     ('svc', SVC(kernel='linear'))
10 ])
11
12 # 训练管道
13 pipeline.fit(X_train, y_train)
14
15 # 保存管道到文件
16 joblib.dump(pipeline, 'pipeline_model.joblib')
```

加载管道:

```
1
2 # 加载管道
3 loaded_pipeline = joblib.load('pipeline_model.joblib')
4
5 # 使用加载的管道进行预测
6 y_pred = loaded_pipeline.predict(X_test)
7
8 # 打印预测结果
9 print("Predictions:", y_pred)
```

管道保存和加载的过程与单一模型相同，只需确保保存和加载整个管道对象即可。

5、模型版本管理

在机器学习的实际应用中，模型的更新和版本管理至关重要。每次训练模型并保存时，最好为模型文件命名加上时间戳或版本号，以便区分不同版本的模型。例如：

```
1
2 import time
3
4 # 创建时间戳
5 timestamp = time.strftime("%Y%m%d-%H%M%S")
6
7 # 保存带时间戳的模型
8 joblib.dump(model, f'svm_model_{timestamp}.joblib')
```

这样，我们可以根据时间戳来管理不同版本的模型，便于模型的回溯和更新。

6、使用模型进行持久化

一旦模型训练完成并保存，我们可以在后续的实际应用中加载该模型来进行预测，而无需重新训练。

例如，我们可以将保存的模型与 Web 服务、批处理作业或其他应用程序集成，使得模型可以反复使用，而无需重新训练。

Web 服务中使用加载的模型

例如，假设我们正在使用 Flask 创建一个简单的 Web 服务，通过 API 接口提供模型预测服务。在这种情况下，我们可以加载保存的模型进行实时预测。

```
1
2 from flask import Flask, request, jsonify
3 import joblib
4 import numpy as np
5
6 app = Flask(__name__)
7
8 # 加载模型
9 model = joblib.load('svm_model.joblib')
10
11 @app.route('/predict', methods=['POST'])
12 def predict():
13     data = request.get_json() # 获取输入数据
```

```
14     features = np.array(data['features']).reshape(1, -1) # 转换成适合预测的格式
15     prediction = model.predict(features) # 使用加载的模型进行预测
16     return jsonify({'prediction': prediction.tolist()}) # 返回预测结果
17
18 if __name__ == '__main__':
19     app.run(debug=True)
```

Sklearn 应用案例

鸢尾花数据集 (Iris Dataset) 是机器学习中最经典的入门数据集之一。

鸢尾花数据集包含了三种鸢尾花 (Setosa、Versicolor、Virginica) 每种花的 4 个特征：花萼长度、花萼宽度、花瓣长度和花瓣宽度。

接下来我们的任务是基于这些特征来预测鸢尾花的种类。

本章节案例将涵盖数据加载、可视化、特征选择、数据预处理、建立分类模型、模型评估及优化等步骤。

1、数据加载与可视化

数据加载

首先，加载鸢尾花数据集，scikit-learn 提供了直接加载鸢尾花数据集的接口。

```
1 from sklearn.datasets import load_iris
2 import pandas as pd
3
4 # 加载鸢尾花数据集
5 data = load_iris()
6
7 # 转换为 DataFrame 方便查看
8 df = pd.DataFrame(data.data, columns=data.feature_names)
9 df['target'] = data.target
10 df['species'] = df['target'].apply(lambda x: data.target_names[x])
11
12 # 查看前几行数据
13 print(df.head())
```

输出结果：

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

此时，数据已经成功加载，并且我们可以看到每条数据的特征和对应的花卉种类。

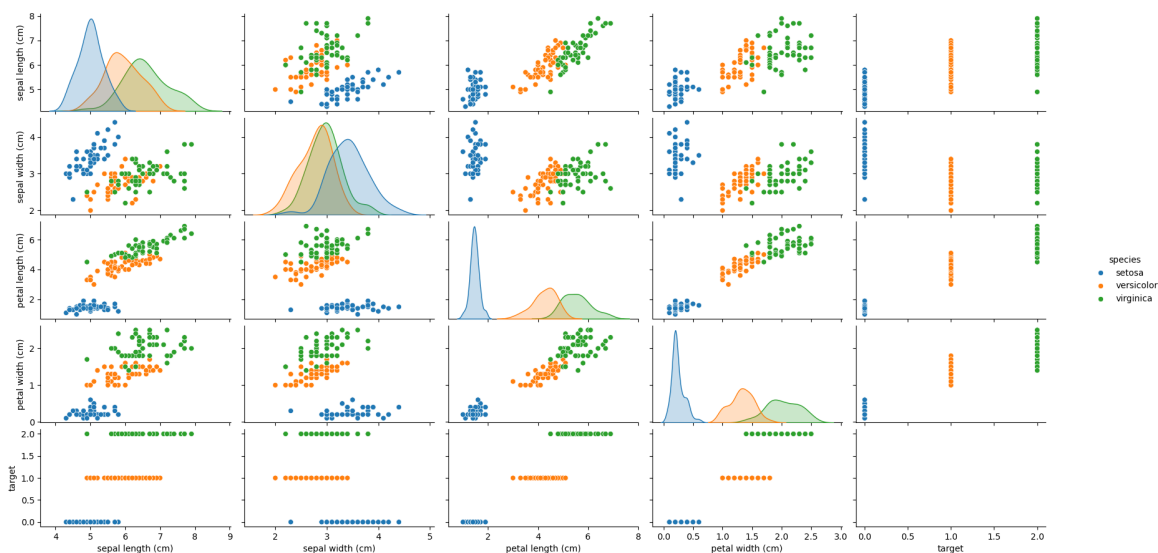
数据可视化

为了更好地理解数据，可以通过可视化的方式查看不同特征之间的关系，我们可以使用 matplotlib 和 seaborn 库来进行可视化。

```
1 from sklearn.datasets import load_iris
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 # 加载鸢尾花数据集
7 data = load_iris()
8
9 # 转换为 DataFrame 方便查看
10 df = pd.DataFrame(data.data, columns=data.feature_names)
11 df['target'] = data.target
12 df['species'] = df['target'].apply(lambda x: data.target_names[x])
13
14 # 绘制特征之间的关系
15 sns.pairplot(df, hue="species")
16 plt.show()
```

pairplot 会绘制特征之间的散点图矩阵，使用不同颜色标识不同的鸢尾花种类，这有助于我们了解各特征的分布和它们之间的关系。

显示图如下：



热力图可视化特征之间的相关性

通过热力图可以查看特征之间的相关性，较强的相关性可以帮助我们在建模时做出更好的选择。

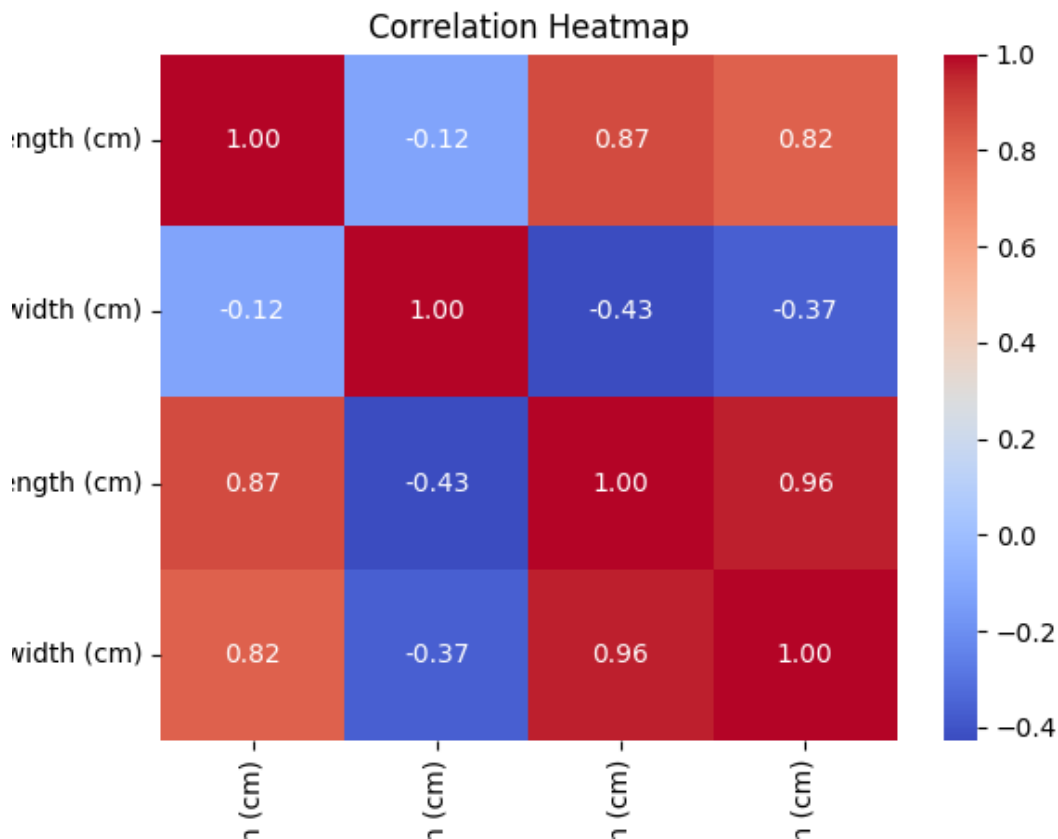
```
1 from sklearn.datasets import load_iris
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 # 加载鸢尾花数据集
7 data = load_iris()
8
```

```

9 # 转换为 DataFrame 方便查看
10 df = pd.DataFrame(data.data, columns=data.feature_names)
11 df['target'] = data.target
12 df['species'] = df['target'].apply(lambda x: data.target_names[x])
13
14 # 绘制特征之间的关系
15 correlation_matrix = df.drop(columns=['target', 'species']).corr()
16 sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
17 plt.title("Correlation Heatmap")
18 plt.show()

```

显示图如下：



2、特征选择与数据预处理

数据预处理

在机器学习中，数据预处理是非常重要的一步。

对于鸢尾花数据集，特征值已经是数值型数据，不需要太多的预处理。但是，我们可以对数据进行标准化，以提高模型的训练效果。

```
1 from sklearn.preprocessing import StandardScaler
2
3 # 提取特征和标签
4 x = df.drop(columns=['target', 'species'])
5 y = df['target']
6
7 # 标准化特征
8 scaler = StandardScaler()
9 x_scaled = scaler.fit_transform(x)
```

标准化的目的是使每个特征的均值为 0，方差为 1，这对于一些基于距离的模型（如 KNN、SVM）非常重要。

特征选择

虽然鸢尾花数据集的特征比较简单，但在实际问题中，有时我们需要通过特征选择来减少特征维度，提升模型效果。

我们可以使用 SelectKBest 或 Recursive Feature Elimination (RFE) 等方法。

例如，使用 SelectKBest 选择与标签最相关的 2 个特征：

```
1 from sklearn.feature_selection import SelectKBest, f_classif
2
3 # 使用卡方检验选择 2 个最相关的特征
4 selector = SelectKBest(f_classif, k=2)
5 x_new = selector.fit_transform(x_scaled, y)
6
7 # 打印选择的特征
8 selected_features = selector.get_support(indices=True)
9 print("Selected features:", x.columns[selected_features])
```

这将选出与目标标签相关性最高的 2 个特征。

3、建立一个分类模型：使用决策树或 SVM 进行分类

使用决策树分类器

我们将首先尝试使用决策树（Decision Tree）模型进行分类。

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.metrics import accuracy_score
4
5 # 划分数据集
6 x_train, x_test, y_train, y_test = train_test_split(x_scaled, y,
7                                                    test_size=0.2, random_state=42)
8
9 # 初始化决策树分类器
10 model_dt = DecisionTreeClassifier(random_state=42)
11
12 # 训练模型
13 model_dt.fit(x_train, y_train)
14
15 # 预测
```

```
15 y_pred_dt = model_dt.predict(X_test)
16
17 # 评估模型
18 accuracy_dt = accuracy_score(y_test, y_pred_dt)
19 print(f"Decision Tree Accuracy: {accuracy_dt:.4f}")
```

使用支持向量机（SVM）进行分类

接下来，我们可以尝试使用支持向量机（SVM）进行分类。

```
1 from sklearn.svm import SVC
2
3 # 初始化 SVM 分类器
4 model_svm = SVC(kernel='linear', random_state=42)
5
6 # 训练模型
7 model_svm.fit(X_train, y_train)
8
9 # 预测
10 y_pred_svm = model_svm.predict(X_test)
11
12 # 评估模型
13 accuracy_svm = accuracy_score(y_test, y_pred_svm)
14 print(f"SVM Accuracy: {accuracy_svm:.4f}")
```

4、评估模型并优化

模型评估

除了准确率外，我们还可以使用其他评估指标，如混淆矩阵、精度、召回率和 F1 分数等。

```
1 from sklearn.metrics import classification_report, confusion_matrix
2
3 # 混淆矩阵
4 cm = confusion_matrix(y_test, y_pred_dt)
5 print("Confusion Matrix (Decision Tree):")
6 print(cm)
7
8 # 精度、召回率、F1 分数
9 report = classification_report(y_test, y_pred_dt)
10 print("Classification Report (Decision Tree):")
11 print(report)
```

网格搜索调优

为了优化模型，我们可以使用网格搜索（GridSearchCV）对模型的超参数进行调优，找到最佳的参数组合。

```
1 from sklearn.model_selection import GridSearchCV
2
3 # 定义决策树的参数网格
4 param_grid = {
5     'max_depth': [3, 5, 10, None],
6     'min_samples_split': [2, 5, 10],
```

```

7     'min_samples_leaf': [1, 2, 4]
8 }
9
10 # 初始化 GridSearchCV
11 grid_search =
    GridSearchCV(estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid, cv=5)
12
13 # 训练网格搜索
14 grid_search.fit(X_train, y_train)
15
16 # 获取最佳参数和最佳模型
17 print("Best Parameters:", grid_search.best_params_)
18 best_model = grid_search.best_estimator_
19
20 # 预测和评估
21 y_pred_optimized = best_model.predict(X_test)
22 accuracy_optimized = accuracy_score(y_test, y_pred_optimized)
23 print(f"Optimized Decision Tree Accuracy: {accuracy_optimized:.4f}")

```

通过网格搜索，我们可以找到最适合当前数据的决策树参数，并提升模型的预测准确率。

交叉验证

为了进一步评估模型的稳定性，我们可以使用交叉验证来评估模型的性能。

```

1
2 from sklearn.model_selection import cross_val_score
3
4 # 进行 5 折交叉验证
5 cross_val_scores = cross_val_score(best_model, X_scaled, y, cv=5)
6 print(f"Cross-validation Scores: {cross_val_scores}")
7 print(f"Mean CV Accuracy: {cross_val_scores.mean():.4f}")

```

交叉验证可以帮助我们评估模型在不同数据子集上的表现，避免模型过拟合。

5、完整代码

以下是一个完整的代码案例，涵盖了鸢尾花数据集的加载、数据预处理、特征选择、建立分类模型、模型评估与优化等步骤。我们将使用决策树和 SVM 分类器，并通过网格搜索优化模型超参数。

```

1
2 # 导入必要的库
3 import numpy as np
4 import pandas as pd
5 import seaborn as sns
6 import matplotlib.pyplot as plt
7 from sklearn.datasets import load_iris
8 from sklearn.model_selection import train_test_split, GridSearchCV,
    cross_val_score
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.svm import SVC
12 from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix

```

```
13
14 # 1. 数据加载
15 # 加载鸢尾花数据集
16 data = load_iris()
17
18 # 转换为 DataFrame 方便查看
19 df = pd.DataFrame(data.data, columns=data.feature_names)
20 df['target'] = data.target
21 df['species'] = df['target'].apply(lambda x: data.target_names[x])
22
23 # 查看前几行数据
24 print("数据预览: ")
25 print(df.head())
26
27 # 2. 数据可视化
28 # 绘制特征之间的关系
29 sns.pairplot(df, hue="species")
30 plt.show()
31
32 # 绘制热力图查看特征之间的相关性
33 correlation_matrix = df.drop(columns=['target', 'species']).corr()
34 sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
35 plt.title("Correlation Heatmap")
36 plt.show()
37
38 # 3. 特征选择与数据预处理
39 # 提取特征和标签
40 X = df.drop(columns=['target', 'species'])
41 y = df['target']
42
43 # 数据标准化
44 scaler = StandardScaler()
45 X_scaled = scaler.fit_transform(X)
46
47 # 4. 建立分类模型
48 # 划分数据集
49 X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
50 test_size=0.2, random_state=42)
51
52 # 使用决策树分类器
53 model_dt = DecisionTreeClassifier(random_state=42)
54 model_dt.fit(X_train, y_train)
55
56 # 预测
57 y_pred_dt = model_dt.predict(X_test)
58
59 # 输出决策树的准确率
60 accuracy_dt = accuracy_score(y_test, y_pred_dt)
61 print(f"Decision Tree Accuracy: {accuracy_dt:.4f}")
62
63 # 使用支持向量机 (SVM) 分类器
64 model_svm = SVC(kernel='linear', random_state=42)
65 model_svm.fit(X_train, y_train)
66
67 # 预测
68 y_pred_svm = model_svm.predict(X_test)
69
70 # 输出SVM的准确率
```

```

70 accuracy_svm = accuracy_score(y_test, y_pred_svm)
71 print(f"SVM Accuracy: {accuracy_svm:.4f}")
72
73 # 5. 模型评估
74 # 决策树模型评估
75 print("\nDecision Tree Classification Report:")
76 print(classification_report(y_test, y_pred_dt))
77
78 print("\nDecision Tree Confusion Matrix:")
79 print(confusion_matrix(y_test, y_pred_dt))
80
81 # SVM模型评估
82 print("\nSVM Classification Report:")
83 print(classification_report(y_test, y_pred_svm))
84
85 print("\nSVM Confusion Matrix:")
86 print(confusion_matrix(y_test, y_pred_svm))
87
88 # 6. 网格搜索调优
89 # 定义决策树的参数网格
90 param_grid = {
91     'max_depth': [3, 5, 10, None],
92     'min_samples_split': [2, 5, 10],
93     'min_samples_leaf': [1, 2, 4]
94 }
95
96 # 初始化 GridSearchCV
97 grid_search =
GridSearchCV(estimator=DecisionTreeClassifier(random_state=42),
param_grid=param_grid, cv=5)
98 grid_search.fit(X_train, y_train)
99
100 # 获取最佳参数和最佳模型
101 print("\nBest Parameters from GridSearchCV (Decision Tree):")
102 print(grid_search.best_params_)
103
104 # 使用最佳模型进行预测
105 best_model = grid_search.best_estimator_
106 y_pred_optimized = best_model.predict(X_test)
107
108 # 输出优化后的决策树准确率
109 accuracy_optimized = accuracy_score(y_test, y_pred_optimized)
110 print(f"Optimized Decision Tree Accuracy: {accuracy_optimized:.4f}")
111
112 # 7. 交叉验证
113 # 进行 5 折交叉验证
114 cross_val_scores = cross_val_score(best_model, X_scaled, y, cv=5)
115 print("\nCross-validation Scores (Optimized Decision Tree):")
116 print(cross_val_scores)
117 print(f"Mean CV Accuracy: {cross_val_scores.mean():.4f}")

```

代码解析：

1. 数据加载：

- 使用 `load_iris()` 加载鸢尾花数据集，并将数据转换为 `DataFrame` 格式，以便查看和分析。

2. 数据可视化：

- 使用 `seaborn` 的 `pairplot` 绘制各特征之间的散点图矩阵，并通过 `heatmap` 绘制特征之间的相关性热力图。
3. 特征选择与数据预处理:
- 提取特征 (`x`) 和标签 (`y`)，并对特征数据进行标准化处理，使得每个特征的均值为 0，方差为 1。
4. 建立分类模型:
- 使用 `DecisionTreeClassifier` 和 `SVC` 分别训练决策树分类器和支持向量机分类器，并评估它们在测试集上的准确率。
5. 模型评估:
- 使用 `classification_report` 和 `confusion_matrix` 输出模型的详细评估指标，包括精度、召回率、F1 分数以及混淆矩阵。
6. 网格搜索调优:
- 使用 `GridSearchCV` 对决策树模型进行超参数调优，寻找最佳的超参数组合，并输出优化后的模型准确率。
7. 交叉验证:
- 使用 `cross_val_score` 进行 5 折交叉验证，评估优化后的决策树模型的稳定性和表现。

输出如下:

1	数据预览:				
2	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target species
3	0	5.1	3.5	1.4	0.2
	0 setosa				
4	1	4.9	3.0	1.4	0.2
	0 setosa				
5	2	4.7	3.2	1.3	0.2
	0 setosa				
6	3	4.6	3.1	1.5	0.2
	0 setosa				
7	4	5.0	3.6	1.4	0.2
	0 setosa				
8					
9	Decision Tree Accuracy: 1.0000				
10	SVM Accuracy: 1.0000				
11					
12	Decision Tree Classification Report:				
13		precision	recall	f1-score	support
14					
15	0	1.00	1.00	1.00	9
16	1	1.00	1.00	1.00	8
17	2	1.00	1.00	1.00	8
18					
19	accuracy			1.00	25
20	macro avg	1.00	1.00	1.00	25
21	weighted avg	1.00	1.00	1.00	25
22					
23	Decision Tree Confusion Matrix:				
24	[[9 0 0]				
25	[0 8 0]				
26	[0 0 8]]				
27					
28	SVM Classification Report:				
29		precision	recall	f1-score	support

```
30
31         0         1.00         1.00         1.00         9
32         1         1.00         1.00         1.00         8
33         2         1.00         1.00         1.00         8
34
35     accuracy                1.00         25
36     macro avg              1.00         1.00         1.00         25
37     weighted avg          1.00         1.00         1.00         25
38
39     SVM Confusion Matrix:
40     [[9 0 0]
41      [0 8 0]
42      [0 0 8]]
43
44     Best Parameters from GridSearchCV (Decision Tree):
45     {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}
46
47     Optimized Decision Tree Accuracy: 1.0000
48
49     Cross-validation Scores (Optimized Decision Tree):
50     [1.         1.         1.         1.         1.         ]
51     Mean CV Accuracy: 1.0000
```